

Assignment 2

Guidelines:

1. Logistics. Your solution must be submitted as a *typed* PDF document. Scanned handwritten solutions, solutions in any other format, or unreadable solutions will not be accepted. You are encouraged to use L^AT_EX.

Please submit a separate PDF for each question.

Submit your solution in Crowdmark by **Friday, Apr 4, 23:59**. If you email the instructor before the deadline to ask for an extension, an extension may be given until **Tuesday, Apr 8, 23:59**. You are strongly encouraged to submit the solution *several days in advance* – last-minute technical problems in your systems and/or in Crowdmark will not grant any extension.

2. Answering questions. In all assignments and exams in this course, unless specifically and explicitly stated otherwise:

- Establish correctness for all of your answers using **rigorous proofs**. Your solution will be marked based on its completeness and correctness as well as on the correctness, clarity and precision of your proofs.
- Describe algorithms in high-level pseudocode. Do not provide low-level executable code. On the other hand, if your code is overly abstract and does not sufficiently specify relevant parts (e.g., it is not obvious how to efficiently implement parts), you may lose marks.
- When you are asked to prove an upper bound on the running time of an algorithm, the intention is to prove the tightest upper bound that you can. Non-tight upper bounds may receive partial (or no) marks.

3. Academic integrity. Assignments are to be solved individually, and according to the course integrity policy and the Code of Behaviour on Academic Matters. Here is a reminder of one of the relevant parts from the syllabus:

Large assignments should be solved individually, and written by yourself, without imitating online texts and without making use of generative AI (or any other software or automated tool). Any submitted assignment that is suspiciously similar to another assignment or to online texts will be automatically investigated as an integrity violation.

You are encouraged to ask questions on Piazza, without directly asking how to solve a problem or revealing the solution. You may not consult any other resources except your class notes, your textbook, Piazza, and assigned readings. Consulting any other resource or collaborating with other students is a violation of the academic integrity policy.

Question 1. We are given a directed weighted graph $G = (V, E)$, where each edge has weight either 1 or 2, and a source node $s \in V$. Our goal is to find the shortest paths from s to any other node in the graph.

1. What is the fastest algorithm we learned for this problem? What is its runtime?
2. Show a faster algorithm (relying on the fact that we're dealing with the special case in which weights are either 1 or 2).

(10pts.)

Answer 1. Dijkstra's algorithm solves the problem in time $O(|E| + |V| \cdot \log(|V|))$.

A faster algorithm works as follows. We first modify G into another graph G' wherein each edge $e = (u, v)$ in G with weight 2 is replaced by a new vertex uv and two edges (u, uv) and (uv, v) . The number of vertices in G' is at most $|V| + |E|$ (since each edge in G contributes at most one new vertex to G') and the number of edges in G' is at most $2|E|$ (since each edge in G is replaced by at most two edges in G'). We run BFS from s on G' and return the (unweighted) distances reported by the BFS algorithm. The complexity of BFS is linear in the number of vertices and edges of G' , so it is at most $O(|V| + |E| + 2|E|) = O(|V| + |E|)$.

To prove correctness, we argue that for every vertex t in G , there is an s - t path of weight r in G if and only if there is an s - t path of length r in G' .

1. To show the $\implies$ direction, we transform any path $P = (s = v_0, v_1, v_2, \dots, v_{r'} = t)$ in G of weight r in G into an s - t path P' of length r in G' . The path P' starts at s . For each edge $e = (v_i, v_{i+1})$ in P , if the weight of e in G is 1 we add the same edge to P' (the edge exists in G' , by construction). If the weight of e in G is 2, we add the two edges $(v_i, v_i v_{i+1})$ and $(v_i v_{i+1}, v_{i+1})$ in G' .

For every $i \in \{0, \dots, r'\}$, after i edge replacements, the path P' reaches vertex v_i and its length is $m_{i,1} + 2m_{i,2}$ where $m_{i,j}$ is the number of edges of weight j in the subpath of P from s to v_i . (The proof is a straightforward induction on $i = 0, \dots, r'$: The base case $i = 0$ is true, and for the induction step, if the i^{th} edge in P has weight 1 then P' is added an edge and otherwise P' is added two edges, and in both cases P' reaches v_i .) In particular, after r' edge replacements, we obtain a path P' from s to t of length $\sum_{i \in [t]} \text{weight}(v_{i-1}, v_i) = m_{t,1} + 2m_{t,2} = r$.

2. To show the $\impliedby$ direction, we transform any path $P' = (s = v_0, v_1, \dots, v_r = t)$ in G' of length r into an s - t path P of weight r in G . The path P starts at s . For each edge $e = (v_i, v_{i+1})$ in P' , we consider two cases. If both v_i and v_{i+1} are vertices in G , it must be that the weight of e is 1 (by the construction of G'); then, we add e to P . Otherwise, the edge can either be of the form (u, uv) or of the form (uv, v) (since there are no edges in G' between vertices of the form uv , by construction). In the former case, we add (u, v) to P ; in the latter case, we do nothing.

We claim that for every $i \in \{0, \dots, r\}$, after iterating over i edges of P' , the path P reaches vertex v_i and its weight is $m_{i,1} + 2m_{i,2}$ where $m_{i,1}$ is the number of edges

in P' of the form (u, v) and $m_{i,2}$ is the number of edges in the subpath P' from s to v_i of the form (u, uv) . The proof is by induction: The base case $i = 0$ is immediate, and the key point in the induction step is that when encountering (u, uv) we add the edge (u, v) which has weight two (by construction of G' – we only replace (u, v) by (u, uv) and (uv, v) if (u, v) has weight 2).

Finally, we claim that $m_{r,1} + 2m_{r,2} = r$. To see this, note that every edge of the form (u, uv) in P' must be followed by an edge of the form (uv, v) ; this is because the only outgoing edge from uv is to v , and t is not of the form uv (because t is a vertex in G). Hence, $m_{r,1} + 2m_{r,2}$ counts each edge in P' exactly once.

Question 2. Design an algorithm that solves the All Pairs Shortest Paths problem using dynamic programming. Your algorithm should run in time $O(|V|^3)$. Compare the runtime of your algorithm to the runtime of Johnson's algorithm: On which type of graphs does one algorithm outperform the other? (10pts.)

Answer 2. This is called the Floyd-Warshall algorithm. A description appears in [CLRS, Chapter 25.2]. Its running time always upper bounds the runtime $O(|E| \cdot |V| + |V|^2 \cdot \log(|V|))$ of Johnson's algorithm, and the latter can be strictly smaller when $|E| = o(|V|^2)$ (i.e., for sparse graphs).

Question 3. For a given flow network (G, c, s, t) , we say that a flow f is integer-valued if $f(e) \in \mathbb{N} \cup \{0\}$ for every $e \in E$. Prove that for every flow network¹ there exists a maximum flow that is integer-valued (i.e., the flow achieves the maximum possible flow value, and is also integer-valued). Also prove that there exists a flow network with a maximum flow that is not integer-valued. (5pts.)

Answer 3. When edge weights are integers, Ford-Fulkerson always returns an integer-valued flow (i.e., all edge flows are integers). This is because in each iteration of FF, the value of the new flow increases by the bottleneck of the augmenting path, and the latter must be an integer (since the minimal capacity of any edge in the augmenting path is an integer). Since Ford-Fulkerson returns a flow with maximum value, there exists a maximum flow that is integer-valued.

To see that some networks may also have maximum flows that aren't integer-valued, consider a network wherein s has two outgoing edges of capacity 1 to v_1 and v_2 , respectively, and each of v_1 and v_2 has an outgoing edge with capacity 1 to v_3 , and v_3 has an outgoing edge with capacity 1 to t . Since $\sum_{e \text{ incoming to } t} c(e) = 1$, the maximum flow in this network is at most 1. A flow with fractional values that achieves this maximum sends $1/2$ from s to v_1 and $1/2$ from s to v_2 , then each of v_1, v_2 sends $1/2$ to v_3 , then v_3 sends 1 to t . By inspection, capacities and conservation are respected, and the value is 1.

¹Recall that we only allowed integer capacities in our flow networks.

Question 4. We are given a flow network where all edges have capacity exactly 1, and an integer $k \in \mathbb{N}$ that is at most the maximum flow in the network. Design a fast algorithm that deletes k edges from the graph to obtain another network in which the maximum flow is smaller by exactly k . (20pts.)

Answer 4. Let us start by describing the algorithm:

- 1: **Input:** (G, c, s, t)
- 2: run Ford-Fulkerson to obtain a maximum flow f
- 3: compute the residual network G_f
- 4: compute the set A of vertices reachable from s in the residual network G_f and the set $B = V \setminus A$
- 5: remove k edges outgoing from A to B
- 6: **return** the resulting graph with k edges removed

We first claim that the running time of this algorithm is $O(|V| \cdot |E| + |V|^2 \cdot \log(|V|))$. This is true because:

1. Ford-Fulkerson runs in time $O(|V| \cdot |E| + |V|^2 \cdot \log(|V|))$.
2. Computing the residual network G_f can be done in time at most $O(|V| \cdot |E| + |V|^2 \cdot \log(|V|))$ (a quick way to see this is to recall that this is one step in the Ford-Fulkerson algorithm).
3. Computing reachability in G_f can be in time $O(|V| + |E|)$ using BFS (and recalling that the number of edges in G_f is at most $2|E|$).
4. Finally, to remove k edges, we go over all vertices in A , go over all vertices in B , and if the pair is connected, we remove the edge, stopping after we removed k edges. This can be done in time at most $O(|A| \cdot |B|) = O(|V|^2)$.

To see that this algorithm is correct, consider the s - t cut (A, B) in G . We proved in class that the cut's capacity precisely equals $\bar{v}(f)$. Since each edge has capacity 1, the number of edges from A to B is precisely $\bar{v}(f)$. In particular, since $k \leq \bar{v}(f)$, there exist at least k edges, and thus Step 5 of the algorithm will succeed.

Let G' be the resulting graph. The min-cut-max-flow theorem tells us that the value of the maximum flow in G' is exactly the minimum capacity of a cut in G' . Note that (A, B) is a cut in G' , and its capacity is $\bar{v} - k$ (because the number of edges from A to B is precisely $\bar{v} - k$, and each one has capacity 1). Hence, the max-flow value in G' is at most $\bar{v}(f) - k$. To prove that it equals $\bar{v}(f) - k$ we need to show that there is no cut with lower capacity in G' . Indeed, if there was such a cut (A', B') , then there would have been less than $\bar{v}(f) - k$ edges from A' to B' . After adding back the k edges we removed, we would have obtained a cut (A', B') with less than $\bar{v}(f)$ edges, and in particular with capacity less than $\bar{v}(f)$. By the min-cut-max-flow theorem, this contradicts the existence of the flow f (whose value is $\bar{v}(f)$).

Question 5. Consider the following problem.

We are given a matrix $M \in \mathbb{R}^{n \times m}$. We want to find a vector $x \in \mathbb{R}^n$ with minimum ℓ_2 norm (i.e., minimum value of $\|x\|_2 = \sqrt{\sum_{i \in [n]} x_i^2}$) such that for every column $i \in [m]$ in M it holds that $\sum_{j \in [n]} M_{j,i} \cdot x_j^2 \geq 1$.

Show how to solve the problem above using linear programming:

1. Write a linear program that allows to solve the problem. Prove that the optimal value for this linear program can be converted (by a single arithmetic operation) to the minimum ℓ_2 norm of a solution for the problem above.
2. Convert your linear program to standard form, and explicitly write the resulting matrix and vectors (A, b, c) . Explain why the standard form is equivalent to the original LP.

(15pts.)

Answer 5. The LP is as follows:

- The variables are $y_1, \dots, y_n$.
- We want to minimize $\sum_{i \in [n]} y_i$.
- There are m constraints $\sum_{j \in [n]} M_{j,i} \cdot y_j \geq 1$, for $i \in [m]$; and additional n constraints $y_i \geq 0$, for $i \in [n]$.

The main claim is that there is a feasible vector for the LP with value v if and only if there is a solution for the problem whose ℓ_2 -norm is $\sqrt{v}$. In particular, the LP has optimal value v if and only if the minimum ℓ_2 -norm of solutions to the problem is $\sqrt{v}$.

Thus, it suffices to prove the claim, as follows:

- Given a feasible vector $y \in \mathbb{R}^n$ for the LP with $\sum_i y_i = v$, let $x \in \mathbb{R}^n$ such that $x_i = \sqrt{y_i}$. Note that x is well-defined, since the LP requires the entries of y to be non-negative. We have that $\|x\|_2 = \sqrt{\sum_i x_i^2} = \sqrt{\sum_i y_i} = \sqrt{v}$, and the requirements $\sum_{j \in [n]} M_{j,i} \cdot x_j^2 \geq 1$ (for $i \in [m]$) are satisfied by the constraints of the LP (since y is feasible).
- On the other direction, if there is $x \in \mathbb{R}^n$ with $\|x\|_2 = \sqrt{v}$, let $y \in \mathbb{R}^n$ such that $y_i = x_i^2$. The constraints of the LP are satisfied since the entries of y are non-negative, and since x is a solution to the problem (so $\sum_j M_{i,j} \cdot y_j = \sum_j M_{i,j} \cdot x_j^2 \geq 1$ for all $i \in [m]$). The objective value is $\sum_{i \in [n]} y_i = \|x\|_2^2 = v$.

Finally, we convert the LP to the standard form. The LP above minimizes $c^T \cdot y$, where $c = 1^n$, subject to $M^T \cdot y \geq 1^m$ and $y \geq 0^n$. This is equivalent to maximizing $-c^T \cdot y$ subject to $-M^T \cdot y \leq -1^m$ and $y \geq 0$; hence, the standard form is:

$$A = -M^T, \quad b = \begin{pmatrix} -1 \\ \dots \\ -1 \end{pmatrix}, \quad c = \begin{pmatrix} -1 \\ \dots \\ -1 \end{pmatrix}.$$

Question 6. In the tutorials you saw an LP relaxation for the Vertex Cover problem.

1. Show an explicit graph $G = (V, E)$ such that the value of the optimal solution for the LP relaxation is different than the minimum size of a vertex cover for G .

Definition. We define the Integrality Gap of a minimization-based LP (A, b, c) over $\mathbb{R}^n$ as $\frac{M^{\text{int}}}{M^{\text{frac}}}$, where

$$M^{\text{int}} = \min_{x \in \{0,1\}^n, A \cdot x \leq b} \left\{ c^T \cdot x \right\} ,$$

$$M^{\text{frac}} = \min_{x \in \mathbb{R}^n, A \cdot x \leq b, x \geq 0^n} \left\{ c^T \cdot x \right\} .$$

2. For every $n \in \mathbb{N}$, show a graph with n vertices such that the integrality gap of the vertex cover LP relaxation is at least $2 \cdot (1 - 1/n)$.
3. Let $x \in \mathbb{R}^n$ be a feasible vector for the vertex cover LP relaxation of an given n -vertex graph. Prove that we can round x to a *feasible* vector $y \in \{0,1\}^n$ such that $c^T \cdot y \leq 2c^T \cdot x$.
4. What bounds do Items #2 and #3 give for the integrality gap of the vertex cover LP relaxation?
5. If you want to get an approximation for the minimum vertex cover size, would you use the LP+rounding algorithm from Item #3, or the algorithm you analyzed in Assignment 1? Why?

(25pts.)

Answer 6.

1. Consider a triangle. The minimum vertex cover size is two, since any two vertices touch all edges, and a single vertex misses one edge. On the other hand, the LP has a solution with value $3/2 < 2$. Specifically, in this solution all variables are assigned value $1/2$. Indeed, all constraints are satisfied, since values are non-negative, and since for each $e = (u, v)$ we have $x_u + x_v = 1$.
2. We generalize the triangle example to a clique on n vertices. The minimum vertex cover size is $n - 1$, since any $n - 1$ vertices touch all edges, but any set with at most $n - 2$ vertices misses an edge (between the excluded vertices). However, the LP relaxation has a solution with value $n/2$, by assigning all variables value $1/2$. (The analysis is identical to that of the triangle.) Thus, $M^{\text{int}} = n - 1$ and $M^{\text{frac}} \leq n/2$, meaning that the integrality gap is at least $\frac{n-1}{n/2} = 2 \cdot (1 - 1/n)$.

3. Let $x \in \mathbb{R}^n$ be a solution to the LP relaxation. Let $y \in \{0,1\}^n$ be the rounded version of x defined by $y_i = \begin{cases} 1 & x_i \geq 1/2 \\ 0 & x_i < 1/2 \end{cases}$. The vertex cover is $S = \{v \in V : y_v = 1\}$.

The size of S is $|S| = \sum_{i \in [n]} y_i \leq 2 \cdot \sum_{i \in [n]} x_i$, where we relied on the fact that the LP requires the entries of x to be non-negative. To see that S is a vertex cover, recall that the LP requires, for each $e = (u, v)$, to have $x_u + x_v = 1$. In particular, either x_u or x_v must be at least $1/2$, meaning that either $y_u = 1$ or $y_v = 1$ (or both), hence S contains a vertex that touches e .

4. In Item #2 we saw that the integrality gap is at least $2 \cdot (1 - 1/n)$ and in Item #3 we saw that it is at most 2 (because Item #3 implies that $\min_{x \in \{0,1\}^n, A \cdot x \leq b} \{c^T \cdot x\} \leq 2 \cdot \min_{x \in \mathbb{R}^n, A \cdot x \leq b} \{c^T \cdot x\}$). In particular, for every n -vertex graph, the integrality gap of the vertex cover LP relaxation is between $2 \cdot (1 - 1/n)$ and 2.
5. It is better to use the algorithm from Assignment 1 than the LP+rounding algorithm. Specifically, the former is a simple greedy algorithm, and even with the most naive implementation it runs in time $O(|V| \cdot |E|)$ (and, of course, it can be optimized using suitable data structures). In contrast, solving an LP is a slower process, running in time that is super-quadratic in the input size.

Question 7. Given a bipartite graph $G = (V, E)$, we define:

1. A fractional matching is an assignment of non-negative real numbers to edges $m: E \rightarrow \mathbb{R}^+$ such that for each $v \in V$ we have $\sum_{e \ni v} m(e) \leq 1$. The value of a fractional matching is $\sum_{e \in E} m(e)$.
2. A fractional vertex cover is an assignment of non-negative real numbers to vertices $c: V \rightarrow \mathbb{R}^+$ such that for each $e = (u, v) \in E$ we have $c(u) + c(v) \geq 1$. The value of a fractional vertex cover is $\sum_{v \in V} c(v)$.

Prove that for every bipartite graph, the maximum value of any fractional matching equals the minimum value of any fractional vertex cover. (15pts.)

Answer 7. We will prove this using LP duality. We create an LP for maximum fractional matching as follows:

- The variables are x_e for every $e \in E$.
- We want to maximize $\sum_{e \in E} m(e)$.
- The constraints are $\sum_{e \ni v} m(e) \leq 1$ for every $v \in V$, and in addition non-negativity constraints (i.e., $x \geq 0^{|E|}$).
- In (A, b, c) form, we have $A \in \mathbb{R}^{|V| \times |E|}$ as the matrix whose rows are indexed by vertices, whose columns are indexed by edges, and $A_{v,e} = 1$ if $v \in e$. The vectors $b \in \mathbb{R}^{|V|}$ and $c \in \mathbb{R}^{|E|}$ are the all-1 vectors.

Now let's construct the dual problem of the LP above. Following the definition of dual problems, we have:

- We have a variable for each row of A . Since there are $|V|$ rows, indexed by the vertices $v \in V$, we have a variable-set $\{y_v\}_{v \in V}$.
- We want to minimize $b^T \cdot y$, which equals $\sum_{v \in V} y_v$ (since b is the all-1 vector).
- The constraints are non-negativity constraints ($y \geq 0^{|V|}$) and $A^T \cdot y \geq c$. The meaning of the latter expression is that for every column e of A we add a constraint $\sum_{v \in V} A_{v,e} \cdot y_v \geq c_e$. Recalling that each column corresponds to an edge e and that $A_{v,e} = 1$ iff v is incident to e , and that c is the all-1 vector, the constraint for $e = (u, v)$ is equivalent to $y_u + y_v \geq 1$.

Thus, the dual LP for maximum fractional matching is precisely the LP that represents minimum fractional vertex cover.² By the duality theorem, the value of the maximum fractional matching equals the value of the minimum fractional vertex cover.

Remark 1. In fact, even the non-fractional optimal values for maximum matching and for minimum vertex-cover are equal. This is known as Konig's theorem.

²Note that it is the LP relaxation of vertex cover.